aslan. | ENGINEERING HIRING

TAKE-HOME TASK MOBILE FIRST

Build a *Smart Wallet* App

A React Native (Expo) coding exercise that tests real-world product thinking, clean architecture, and mobile UX craft — with a web target too.

DEADLINE
7 days from receipt

SUBMISSION
GitHub repo

01 Overview

You'll build a mobile-first wallet app with Google sign-in, savings pots, a voucher shop, and a loyalty points scheme. The goal isn't pixel-perfection — we're looking at your decision-making, code structure, and how you handle product complexity under time pressure.

> **TIP**
>
> We'd rather see three features done *really well* — with good error handling, loading states, and clean code — than five features that barely work. Scope it yourself and be transparent in your README about what you cut and why.

02 Features to Build

🔐

Google Sign-In

OAuth 2.0 authentication. Show the user's name and avatar in the app header.

💰

Main Wallet

A balance displayed prominently. All transactions debit/credit this balance.

🪣

Savings Pots

Create named pots. Transfer money in and out. Show a pot-level balance.

🏷️

Voucher Shop

Buy preset vouchers. Deducts from wallet, generates a voucher code.

⭐

Loyalty Points

Earn points on purchases. Redeem points for wallet credit.

2.1 — AUTHENTICATION

- → Sign in with Google (OAuth 2.0). No username/password flows.
- → Persist the session so users don't sign in on every launch.
- → Show the user's Google profile picture and display name in the app.
- → A sign-out option must exist somewhere in the UI.

2.2 — MAIN WALLET

- → Display a running balance, visible on the home screen.
- → A seeded starting balance is fine (e.g. £500.00) — no need for top-up flows.
- → Show a transaction history: date, description, amount, running balance.
- → Balance must never go negative. Show a clear error if a transaction would overdraw it.

2.3 — SAVINGS POTS

- → User can create one or more named pots (e.g. "Holiday", "Emergency Fund").
- → User can add money to a pot (deducted from main wallet).
- → User can withdraw from a pot back to the main wallet.
- → Each pot shows its own balance. The main wallet balance excludes pot money.
- → Optionally: allow deleting a pot (funds return to wallet).

2.4 — VOUCHER SHOP

- → Display at least 3 preset voucher denominations (e.g. £10, £25, £50, £100).
- → Purchasing a voucher deducts the full face value from the main wallet.
- → Generate a short voucher code on purchase (e.g. `VCHX-4K2M`). It doesn't need to be redeemable — just displayed.
- → Purchased vouchers are listed in a "My Vouchers" section.

2.5 — LOYALTY POINTS

Every voucher purchase earns points. Points can be redeemed for wallet credit. Points are earned at one point per whole pound spent. Redemptions are in multiples of 100 points, with 100 points worth £1.00 of wallet credit.

VOUCHER VALUE	POINTS EARNED	POINTS → CREDIT	EFFECTIVE CASHBACK
£10	`10 pts`	`100 pts = £1.00`	1%
£25	`25 pts`	`100 pts = £1.00`	1%
£50	`50 pts`	`100 pts = £1.00`	1%
£100	`100 pts`	`100 pts = £1.00`	1%

- → Show the user's current points balance prominently (e.g. in a profile or rewards screen).
- → Allow the user to redeem points in multiples of 100, up to their full balance.
- → Redeemed credit is added directly to the main wallet.
- → Points balance cannot go negative.

03 Technical Requirements

STACK

- → React Native with Expo. The app must run on both mobile (iOS or Android simulator) and web.
- → TypeScript throughout.
- → State management of your choosing (Zustand, Redux Toolkit, Jotai, etc.). Document your choice briefly in the README.
- → Local persistence for state between sessions (e.g. AsyncStorage, MMKV). A backend is not required.

MOBILE UX

- → Handle loading states and errors gracefully — no unhandled promise rejections.
- → Keyboard avoidance for any input forms (transfer amounts, pot names).

BONUS POINTS

Deploy the web version somewhere publicly accessible (Vercel, Netlify, Expo hosting, etc.) and include the URL in your README

04 What to Submit

GITHUB — **Public GitHub repository**

Clean commit history preferred. Don't squash everything into one commit — we like seeing how you think as you work.

README — **README.md**

Setup instructions, tech choices and reasoning, known limitations, and anything you'd do differently with more time.

OPTIONAL — **Live deployment URL**

If you deploy the web version (Vercel, Netlify, Expo hosting, etc.), include the URL in your README. Not required — we're happy to run it locally from clear instructions.

05 How We'll Evaluate

We review every submission against the same rubric. There are no trick requirements — we're looking for someone who writes code they'd be comfortable maintaining in a year.

Product logic correctness Calculations, edge cases, balance integrity	HIGH WEIGHT
Mobile UX & interaction design Navigation, loading states, error handling	MID WEIGHT
Feature completeness How many of the 5 features work end-to-end	MID WEIGHT
Testing At least one meaningful test suite	MID WEIGHT
README clarity & self-awareness Documenting trade-offs and decisions	LOWER WEIGHT

06 Questions & Clarifications

CAN I USE A UI COMPONENT LIBRARY?

Yes — NativeBase, Tamagui, React Native Paper, etc. are all fine. Just make sure you're demonstrating your own thinking in the layout and not just wiring up pre-built screens.

DO I NEED A REAL BACKEND?

No. Everything can be local (AsyncStorage, SQLite, in-memory with persistence). The Google OAuth flow does require a real client ID — set up a free Firebase or Google Cloud project. Document the setup steps.

WHAT HAPPENS AFTER I SUBMIT?

We'll review your repo, then invite you to an interview where we'll walk through your solution together — your design choices, trade-offs, and how you'd extend it.

WHAT IF I CAN'T FINISH IN TIME?

Submit what you have before the deadline and document what's missing in the README. A partial but well-written submission beats a rushed complete one. Submissions received after day 7 will not be accepted.